

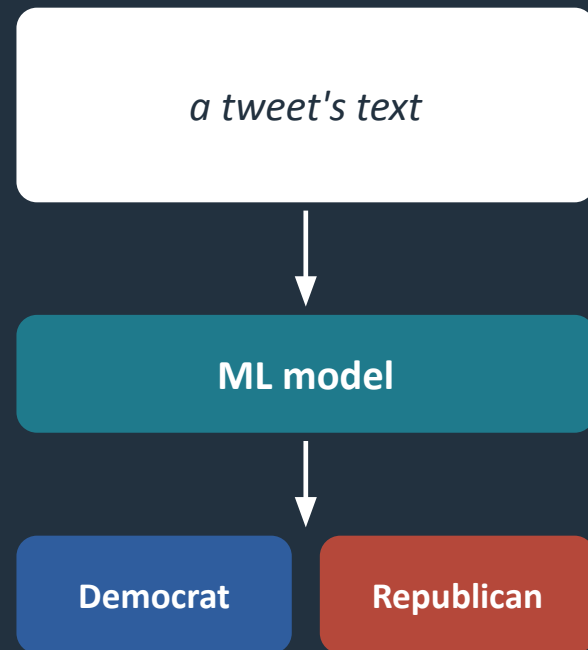
Political Bias Classification

Predicting the party behind US Congress tweets with classical machine learning

Velkur Tanisha Reddy

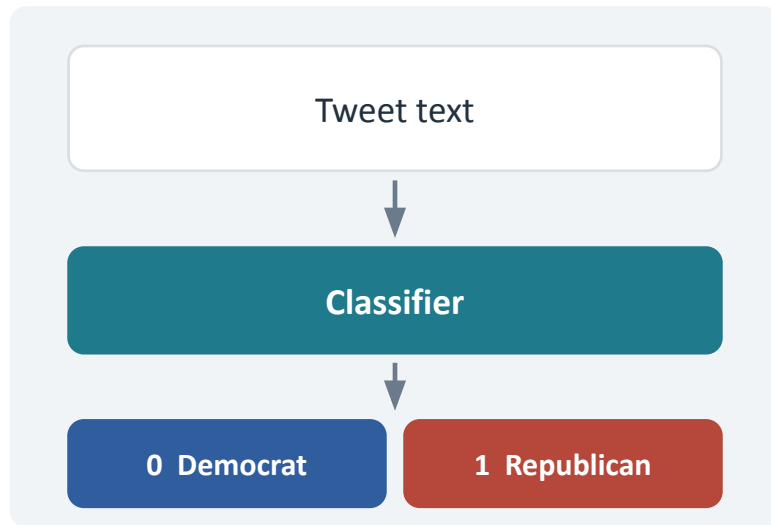
Gagan K C

PES University



Problem statement

- Given the text of a tweet from a US Congress member, can a simple ML model predict the party of the account?
- This is binary text classification. Democrat is class 0 and Republican is class 1.
- Motivated by the Stanford CS229 report on political bias in news articles. We use tweets instead.



What the model predicts: the party of the account that posted the tweet. This is not an independently verified bias score.

Objectives

1

Clean and prepare the political tweet data

2

Build three text preprocessing variants

3

Train and compare three classifiers

4

Select the model using validation macro F1

5

Test the selected model once on unseen data

6

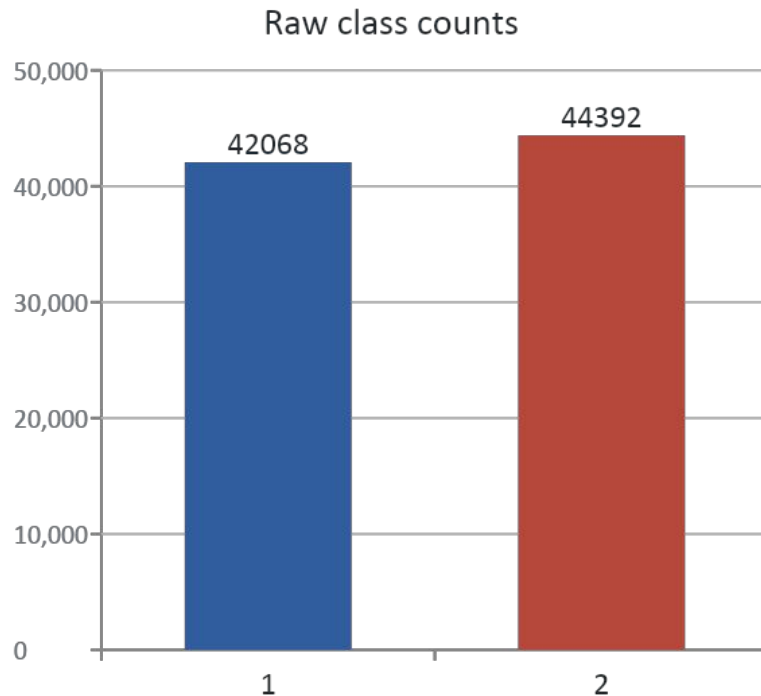
Report results with standard metrics

Dataset and source

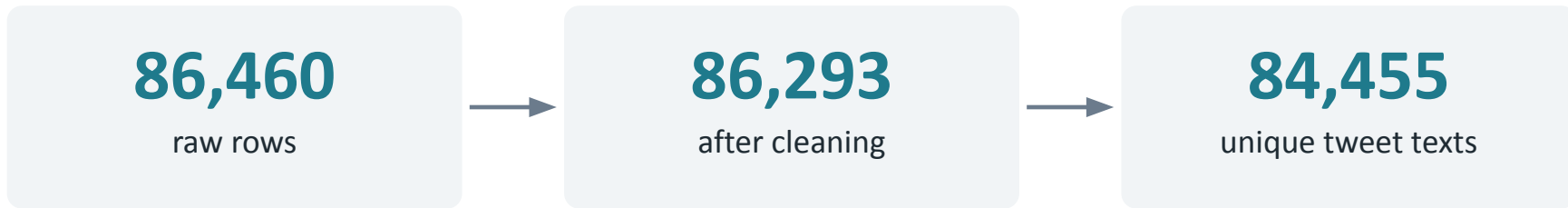
- Democrat vs. Republican Tweets, from Kaggle
- File used is ExtractedTweets.csv
- Columns are Party, Handle and Tweet
- Tweets from members of the US Congress
- Label is the Party column

86,460

raw tweets before any cleaning. Classes are close to balanced.



Preprocessing: cleaning the data



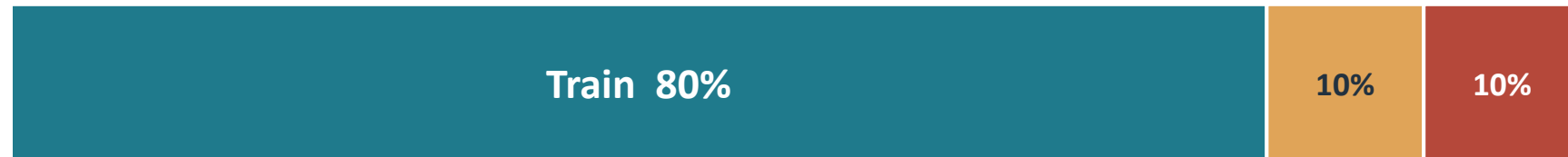
Cleaning steps (notebook 01)

- Drop rows with missing Tweet or Party
- Keep only the labels Democrat and Republican
- Remove tweets whose identical text appears under both parties, since they add label noise
- Drop exact duplicates on Tweet, Party and Handle

Before splitting

- Drop repeated tweet texts globally. This removes another 1,838 rows.
- Retweets and copy-pasted messages are common, so the same text could land in both train and test and inflate the score

Train, validation and test split



● Train 67,564 tweets

The models learn from this.
Vectorizers are fitted here too.

● Validation 8,445 tweets

Used to compare the three
models and pick one.

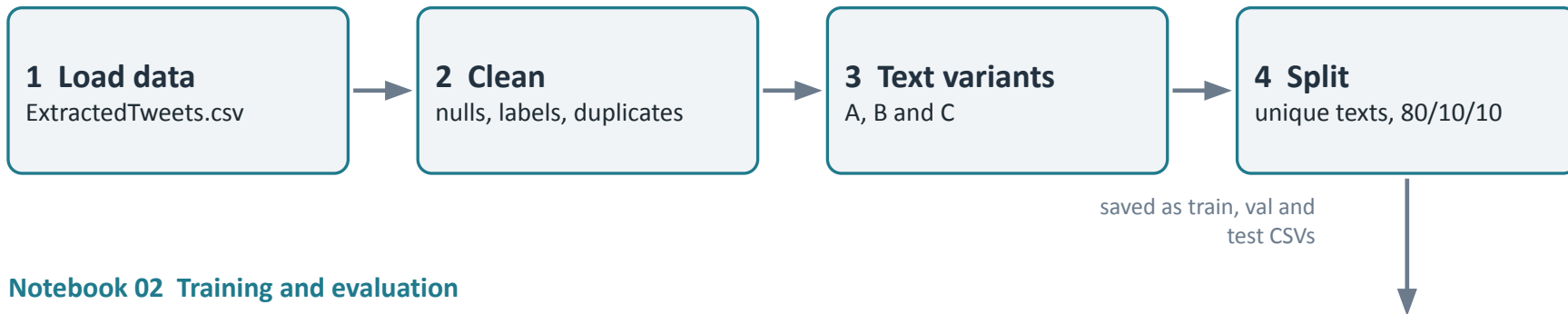
● Test 8,446 tweets

Used once at the end, only for
the selected model.

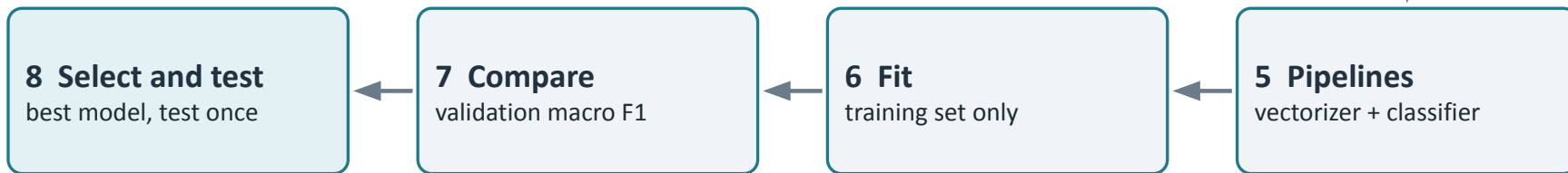
Stratified by Party, so each part keeps the same Democrat and Republican ratio. `random_state=42` so the split can be reproduced.

Project workflow

Notebook 01 Data preprocessing



Notebook 02 Training and evaluation



The vectorizer lives inside each pipeline, so it learns its vocabulary from the training data only.

The three models

Bernoulli NB

Input: Binary word counts (word present or not)

Naive Bayes that looks at whether each word appears. It also uses the absence of words.

Multinomial NB

Input: Raw word counts

Naive Bayes that uses how many times each word appears. The usual choice for text.

LinearSVC

Input: TF-IDF weighted words

Linear SVM that finds a separating line between the classes. TF-IDF reduces the weight of very common words.

```
CountVectorizer(binary=True) + BernoulliNB()  
CountVectorizer() + MultinomialNB()  
TfidfVectorizer() + LinearSVC(random_state=42, dual=False)
```

scikit-learn pipelines from notebook 02. Default settings, no hyperparameter tuning.

Evaluation metrics

Accuracy

Share of all tweets predicted correctly

Macro precision

Of tweets predicted as a class, how many were right. Averaged over both classes.

Macro recall

Of tweets truly in a class, how many we found. Averaged over both classes.

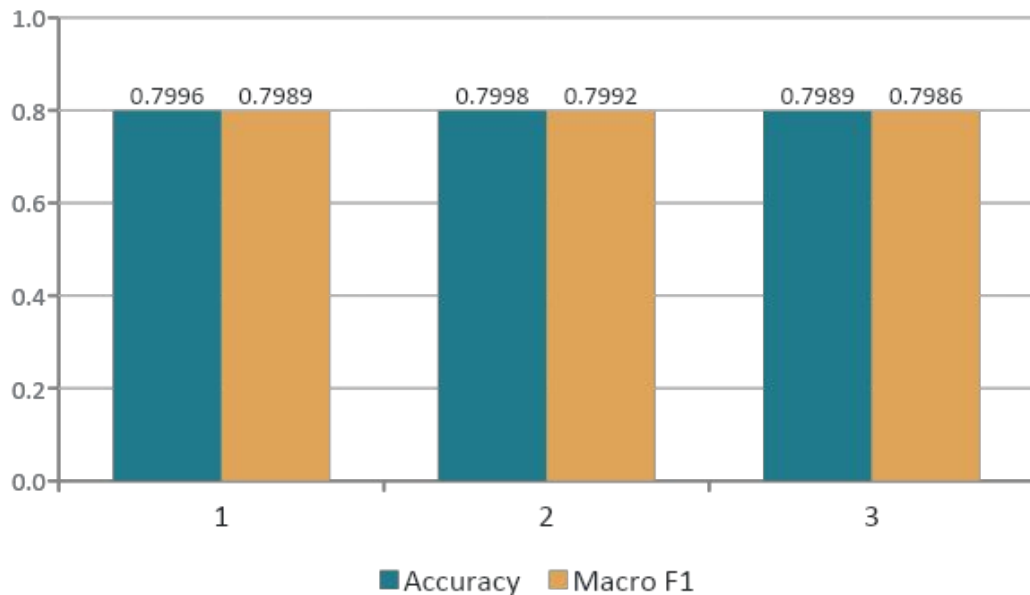
Macro F1

Balance of precision and recall, averaged over both classes. Used for model selection.

Why macro F1?

- Classes are not exactly equal. Validation has 4,106 Democrat and 4,339 Republican tweets.
- Macro averaging treats both parties equally.
- Always guessing Republican would score about 51.4% accuracy ($4,339 / 8,445$). Our models should beat that.

Model comparison on validation set



Per class on validation (precision / recall)

	BNB	MNB	SVC
Dem.	0.81 / 0.79	0.81 / 0.79	0.80 / 0.79
Rep.	0.80 / 0.81	0.80 / 0.81	0.80 / 0.81

- All three land at about 80%
- Gap in macro F1 between best and worst is only 0.0006
- Multinomial NB is highest, but it is close to a tie

Selected model and test results

Why Multinomial Naive Bayes?

- It had the highest validation macro F1 (0.7992)
- The choice was made on validation only, using the 8,445 validation tweets
- The test set was then used exactly once for this model
- The margin over the others is tiny, so treat it as a tie-break

79.94%

Test accuracy

0.7988

Test macro F1

80.01%

Test macro precision

79.86%

Test macro recall

Validation macro F1 was 0.7992 and test macro F1 is 0.7988. They are very close, so we see no sign that the choice overfit the validation set. Guessing the larger class would give roughly 51%.

Conclusion

1 About 80% on unseen data

Multinomial NB reached 79.94% accuracy and 0.7988 macro F1 on the test set, well above guessing.

2 The models are nearly tied

Validation macro F1 differs by only 0.0006 across the three models.

3 Word usage carries the signal

A simple Naive Bayes with default settings matches a linear SVM, so word choice seems to carry most of the information.

4 A careful setup matters

Removing duplicates before splitting and testing only once keeps the score honest.

Limitations and future improvements

Limitations

- Labels are party, not verified political bias
- One dataset, one set of accounts, one time period
- Tweets are short and informal. Results may not carry over to news articles.
- One split and one run, no cross-validation
- Default settings, no tuning. Only text variant B was trained.

Future improvements

- Train and compare text variants A and C against B
- Tune smoothing and regularization with cross-validation
- Try bigrams or word embeddings
- Look at the errors to see which tweets get confused
- Test on real news articles, as in the Stanford paper